Wireshark overview

Wireshark Use Cases

Wireshark is a network traffic analyser used to inspect packets in detail.

Main Uses

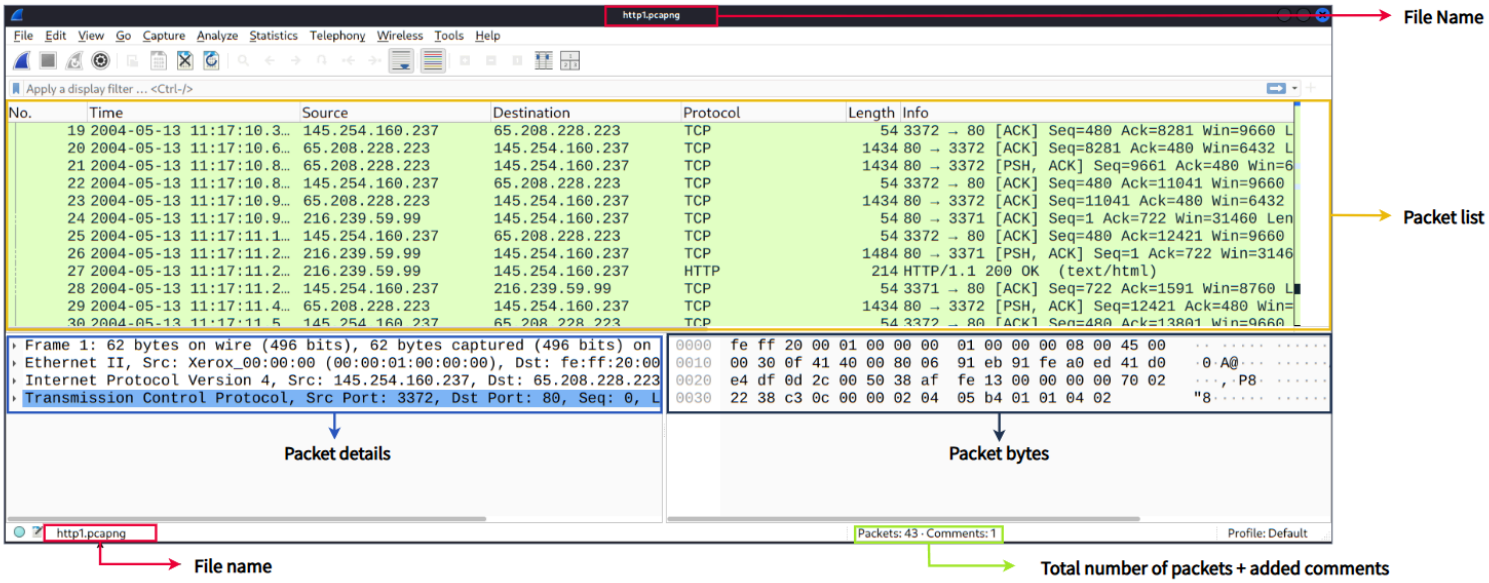
1. Network Troubleshooting
   - Detect network congestion
   - Identify failure points
   - Analyse network load

2. Security Analysis
   - Detect rogue hosts
   - Identify abnormal ports
   - Spot suspicious traffic

3. Protocol Analysis
   - View protocol response codes
   - Inspect payload data
   - Understand how protocols work

Packet Viewing Panels

When a PCAP is loaded, Wireshark shows 3 analysis panes.



1. Packet List Pane

Shows packet summary:

- Source IP
- Destination IP
- Protocol
- Packet info

2. Packet Details Pane

Shows protocol breakdown.

Example:

- ➔ Ethernet
- ➔ Internet Protocol
- ➔ Transmission Control Protocol
- ➔ Hypertext Transfer Protocol

3. Packet Bytes Pane

Shows packet data in:

- Hexadecimal
- ASCII

Useful for:

- Viewing payloads
- Detecting hidden data
- Finding flags

Capture File Details

You can view metadata of a PCAP file.

Statistics → Capture File Properties

Shows:

- File hash
- Capture time
- File comments
- Capture interface
- Packet statistics

Wireshark Packet Dissection

Packet Dissection

Packet dissection (protocol dissection) is the process of:

- Breaking down packets
- Decoding protocols
- Viewing protocol fields

Wireshark automatically interprets protocols inside packets.

Example protocols Wireshark can dissect:

- HTTP
- DNS
- TCP
- UDP
- SMB
- FTP

It can also support custom dissectors via scripts.

Packet Details in Wireshark

To analyse a packet:

1. Click a packet in the Packet List Pane
2. Packet details appear in the Packet Details Pane

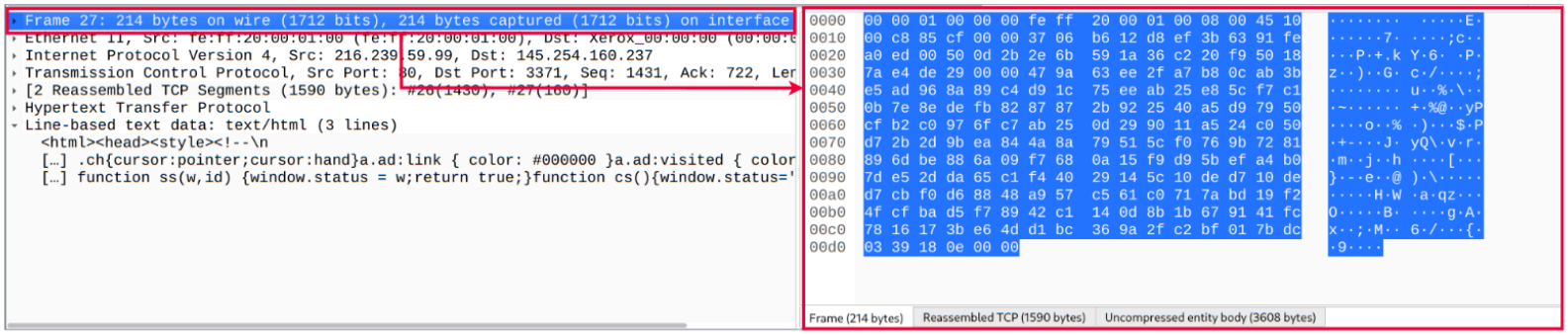
Shortcut:

Double-click packet → Opens in new window

When you click a field:

The related bytes are highlighted in the Packet Bytes Pane

This helps analysts map protocol fields to raw data.



Packet Structure (OSI Model)

Packets typically contain 5–7 layers.

Wireshark displays these layers based on the OSI model.

Typical structure:

1. Frame
2. Ethernet (MAC)
3. IP
4. TCP/UDP
5. Protocol Errors
6. Application Protocol
7. Application Data

Layer 1 — Frame (Physical Layer)

Represents the captured packet itself.

Shows:

- Frame number
- Frame length
- Capture time
- Interface used

```
▾ Frame 38: 478 bytes on wire (3824 bits), 478 bytes captured (3824 bits)
  ▸ Interface id: 0 (unknown)
    Encapsulation type: Ethernet (1)
    Arrival Time: May 13, 2004 10:17:12.158193000 UTC
    [Time shift for this packet: 0.000000000 seconds]
    Epoch Time: 1084443432.158193000 seconds
    [Time delta from previous captured frame: 0.070101000 seconds]
    [Time delta from previous displayed frame: 0.070101000 seconds]
    [Time since reference or first frame: 4.846969000 seconds]
    Frame Number: 38
    Frame Length: 478 bytes (3824 bits)
    Capture Length: 478 bytes (3824 bits)
    [Frame is marked: False]
    [Frame is ignored: False]
    [Protocols in frame: eth:ethertype:ip:tcp:http:data-text-lines:xml]
    [Coloring Rule Name: HTTP]
    [Coloring Rule String: http || tcp.port == 80 || http2]
```

Layer 2 — MAC Addresses (Data Link Layer)

Shows hardware addresses.

Displayed fields:

- Source MAC
- Destination MAC

Example:

```
Src MAC: 00:0c:29:5d:21:8a
Dst MAC: 00:50:56:c0:00:08
```

```
▸ Frame 38: 478 bytes on wire (3824 bits), 478 bytes
▾ Ethernet II, Src: fe:ff:20:00:01:00 (fe:ff:20:00:0
  ▸ Destination: Xerox_00:00:00 (00:00:01:00:00:00)
  ▸ Source: fe:ff:20:00:01:00 (fe:ff:20:00:01:00)
    Type: IPv4 (0x0800)
▸ Internet Protocol Version 4, Src: 65.208.228.223,
▸ Transmission Control Protocol, Src Port: 80, Dst P
▸ [14 Reassembled TCP Segments (18364 bytes): #6(138
▸ Hypertext Transfer Protocol
▸ eXtensible Markup Language
```

Layer 3 — IP Addresses (Network Layer)

Shows logical addressing.

Fields include:

- Source IP
- Destination IP

```
▸ Frame 38: 478 bytes on wire (3824 bits), 478 bytes captured (3824
▸ Ethernet II, Src: fe:ff:20:00:01:00 (fe:ff:20:00:01:00), Dst: Xero
▾ Internet Protocol Version 4, Src: 65.208.228.223, Dst: 145.254.160.
    0100 .... = Version: 4
    .... 0101 = Header Length: 20 bytes (5)
  ▸ Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
    Total Length: 464
    Identification: 0xc0ac (49324)
  ▸ Flags: 0x4000, Don't fragment
    Fragment offset: 0
    Time to live: 47
    Protocol: TCP (6)
    Header checksum: 0x2fe0 [validation disabled]
    [Header checksum status: Unverified]
    Source: 65.208.228.223
    Destination: 145.254.160.237
  ▸ [Source GeoIP: US, ASN 17338, UNITAS-AOS]
  ▸ [Destination GeoIP: Stockelsdorf, DE, ASN 3209, Vodafone GmbH]
▸ Transmission Control Protocol, Src Port: 80, Dst Port: 3372, Seq:
▸ [14 Reassembled TCP Segments (18364 bytes): #6(1380), #8(1380), #1
▸ Hypertext Transfer Protocol
▸ eXtensible Markup Language
```

- TTL
- Packet length

Example:

```
Src IP: 192.168.1.10
Dst IP: 192.168.1.1
```

Layer 4 — Transport Protocol

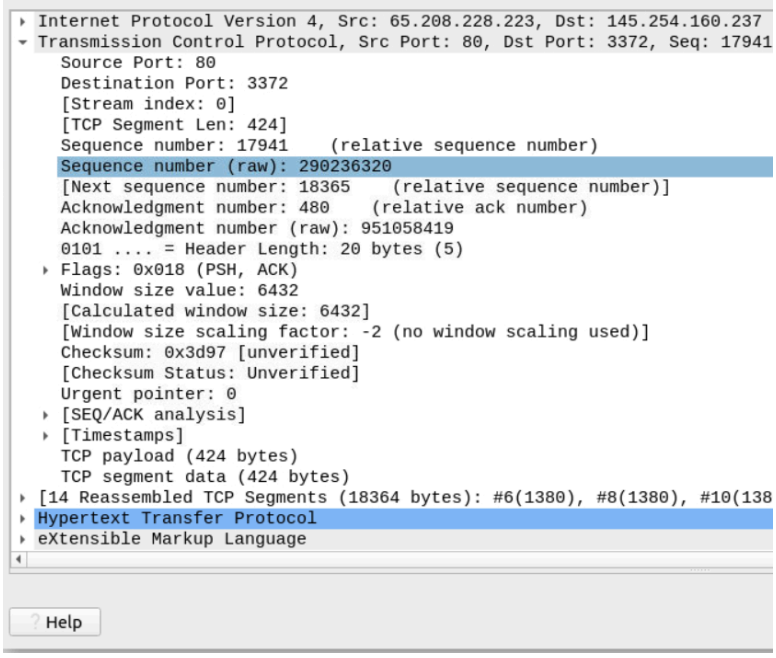
Shows TCP or UDP details.

Important fields:

- Source port
- Destination port
- Sequence number
- Acknowledgement number
- Flags

Example:

```
Src Port: 51532
Dst Port: 80
Protocol: TCP
```

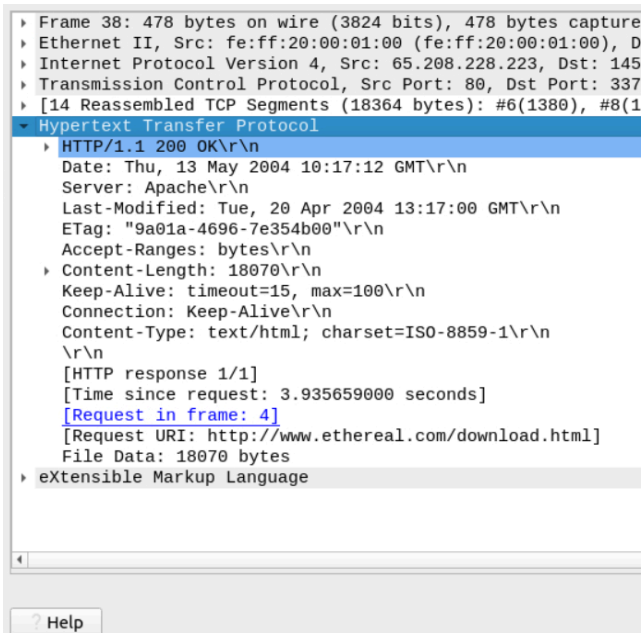


Protocol Errors (TCP Reassembly)

This section appears when:
TCP packets are split across multiple segments

Wireshark will:

- Reassemble TCP segments
- Detect missing segments



Layer 5 — Application Protocol

Shows details of the actual application protocol.

Examples:

HTTP	Web traffic
FTP	File transfer

Example:

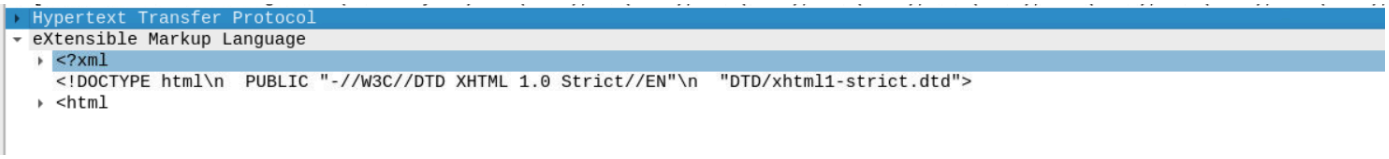
```
GET /login HTTP/1.1
Host: example.com
```

Purpose:
Understand user activity

Application Data
Shows the actual data being transmitted.

Examples:

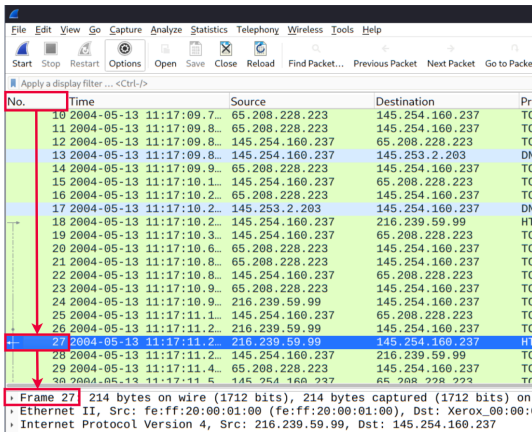
- Login credentials
- Form data
- File contents
- API requests



Packet Navigation & Investigation

Packet Numbers

Wireshark assigns a unique number to every packet in the capture.



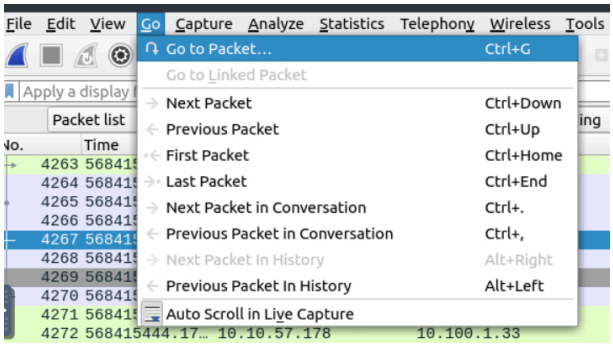
Purpose:

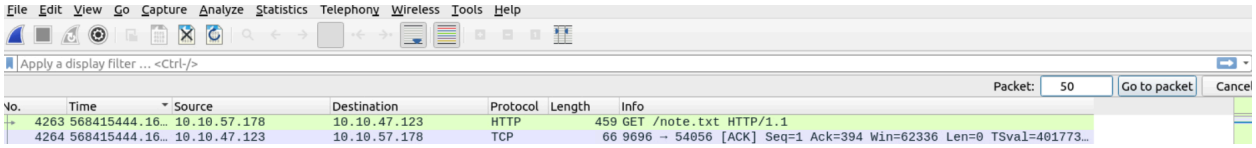
- Helps track packets in large PCAP files
- Makes it easy to jump to a specific packet
- Useful when investigating attack timelines

Go To Packet

The Go To Packet feature allows analysts to navigate quickly.

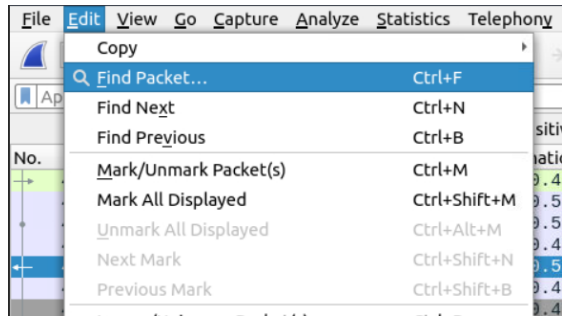
Navigation options:
Jump to specific packet number
Move next / previous packet
Jump to packets in a conversation





Find Packets (Search Feature)

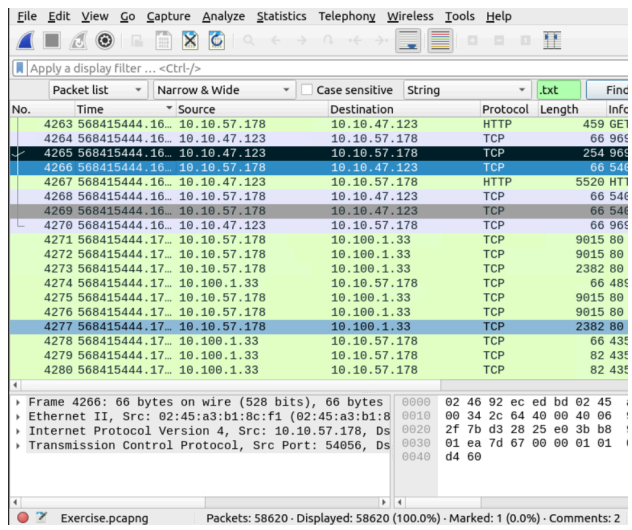
Wireshark can search inside packets for specific content.



Menu:
Edit → Find Packet

Mark Packets

Analysts can mark important packets.



Purpose:

- Highlight suspicious packets
- Return to important events quickly
- Export selected packets later

Menu:
Right Click → Mark Packet

Packet Comments

Packet comments allow analysts to leave investigation notes.

Example comment:

```
Possible credential submission here
```

Benefits:

- Helps team investigations
- Documents suspicious behaviour
- Keeps notes inside the PCAP

Expert Info (Automatic Analysis)

Wireshark has an Expert System that detects potential problems.

Menu:
Analyse → Expert Information
It highlights suspicious events.

Packet Filtering

Packet filtering helps analysts remove unnecessary traffic (noise) and focus on important packets.

Wireshark provides two filtering methods:

Capture Filter	Filters packets while capturing traffic
Display Filter	Filters packets after capture for analysis

Apply as Filter

The Apply as Filter option allows you to filter packets by a specific field value.

Steps:

1. Click a field
2. Right click
3. Select:
    a. Apply as Filter

Conversation Filter

Conversation Filter shows all packets related to a communication session.

It filters based on:

- IP addresses
- Ports

Menu:
Right Click → Conversation Filter

Example use case:

Investigating communication between:
192.168.1.5 ↔ 192.168.1.10

Result:
Only packets related to this conversation appear.

Colourise Conversation

This feature highlights packets in a conversation without filtering them.

Menu:
View → Colourise Conversation

Reset colours:
View → Colourise Conversation → Reset Colourisation

Prepare as Filter
Similar to Apply as Filter, but it does not execute immediately.

Instead:
- The filter query is placed in the filter bar
- Analyst can modify it before running.

Menu:
Right Click → Prepare as Filter

Apply as Column
Wireshark packet list shows limited default columns.
Using Apply as Column, you can add new fields.

Menu:
Right Click → Apply as Column

Example fields to add:
- Source port
- HTTP host
- TCP stream number

This Helps analysts compare values across packets quickly.

Follow Stream
Wireshark splits traffic into packets, but applications communicate in streams.
Follow Stream reconstructs the full conversation.

Menu:
Right Click → Follow → TCP Stream

Other options:
- Follow UDP Stream
- Follow HTTP Stream

Filter by Protocol

Filter by protocol name.

Example:

- http

Other examples:

- dns
- ftp
- smtp
- imap
- arp
- Dhcp

Filter by Port

Structure:
tcp.port == <port>

Example:
tcp.port == 80

Filter by IP Address

Structure:
ip.addr == <IP>

Example:
ip.addr == 192.168.1.2